

Neural Proof Optimization

Riyaz Ahuja

April 20, 2026

Department of Mathematics, Carnegie Mellon University

Outline

Motivation

Proof Optimization

ImProver

ImProver Experiments

ImProver 2

ImProver 2 Experiments

Conclusions

Core Contributions

C1. Task formalization

Proof optimization defined over a verified proof (c, x, y_0) and arbitrary metric μ ; NTP shown to be a strict special case

C2. ImProver: agentic proof optimizer

ICLR 2025

GPT-4o agent with CoS prompting, iterative refinement, and RAG; outperforms naive GPT-4o by 423–778% on undergraduate proofs

C3. ImProver 2: trained small model

ICML 2026 submitted

7B model trained via IRPO self-improvement + neurosymbolic scaffold; outperforms 671B models on structural metrics at a fraction of the cost

C4. Specialization beats scale

Gains track formal-structure exposure (CoS, neurosymbolic augmentation), not model size; proof optimization is a specialization problem

Motivation

Mathematical Collaboration and the PNT

Theorem

The number of primes less than n grows as $O(n/\ln n)$

- Proven independently by Hadamard and Poussin in 1896
- However, conjectured nearly 100 years prior (Gauss, Legendre)
- What was the gap?

Infrastructure:

- Hadamard, Poussin both employed complex analytic techniques.
- Without 100 years of development of the shared infrastructure on the zeta function, contour integration, analytic continuation, etc., the proof would have been impossible.

Despite the importance of shared public infrastructure and library development in mathematics, traditional collaboration is fundamentally limited by a *trust bottleneck*.

However, proof assistants allow for the decentralization of trust: anyone, anywhere can contribute as their correctness is formally checked and verified. This allows for larger scale collaboration.

Proof Assistants

Lean 4 is one such proof assistant, with an especially large and robust infrastructure library.

- Proofs are programs: Lean's type system enforces logical correctness
- Each tactic transforms the proof state: the current hypotheses and goals
- A small, trusted kernel checks every step

```
-- set intersection distributes over union
example :  $s \cap (t \cup u) =$ 
            $(s \cap t) \cup (s \cap u)$  := by
  ext x
  simp only [Set.mem_inter_iff,
             Set.mem_union]
  tauto
```

Proof Assistants Enable Decentralized Trust

- Lean 4: a proof assistant where proofs are checked by a small, audited **kernel**
- You trust the kernel, not the author
- Anyone, anywhere can contribute — correctness is checked mechanically
- This enables **trust-free, decentralized formal mathematics at scale**

A formal proof is a machine-checkable certificate, not a social agreement.

What a Lean Proof Looks Like

- Proofs are programs: Lean's type system enforces logical correctness
- Each *tactic* transforms the proof state: the current hypotheses and goals
- The kernel checks every step

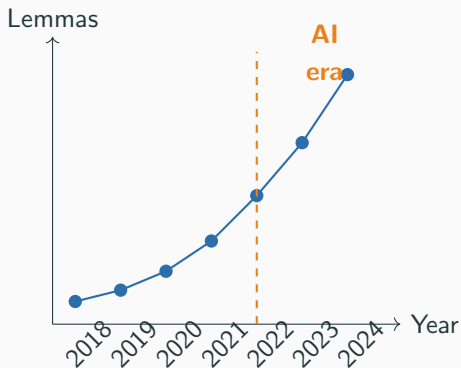
```
-- set intersection distributes over union
example : s ∩ (t ∪ u) =
    (s ∩ t) ∪ (s ∩ u) := by
  ext x
  simp only [Set.mem_inter_iff,
             Set.mem_union]
  tauto
```

Mathlib: Formal Mathematics at Scale

- Lean's community mathematics library
- $> 100,000$ formal lemmas: algebra, topology, analysis, number theory, and more
- Hundreds of active contributors worldwide
- **Growing by thousands of lemmas per month**

Lean's trust-free model makes this scale possible.

Mathlib is Growing Exponentially



Growth already outpaces what human reviewers can reliably curate.

But: Review is Still Manual

- Every Mathlib PR is reviewed by human maintainers
- Reviewers check: mathematical correctness, style, documentation, robustness
- Contributions must be rewritten until they meet the library's style guidelines
- As the library grows, inter-dependencies make reviews harder

Even today, the growth rate already stresses human capacity.

Is Proof Generation Being Automated?

- **Neural Theorem Proving (NTP)**: use language models to write Lean proofs automatically
- The field has exploded since 2020
- Two parallel questions for formal math:
 - Can AI *generate* proofs? **Yes, rapidly improving**
 - Can AI *review and improve* proofs? **This thesis**

NTP: Search-Based Provers (2020–2022)

- **GPT-f (2020)**: first large-scale neural theorem prover; tactic-by-tactic generation
- **HyperTree Proof Search (2022)**: MCTS online self-training; first competition-level results
- **BFS-Prover (2022)**: breadth-first search with learned value function

Treat proving as a *search problem*: explore tactic space with a model as heuristic.

NTP: LLM-Based and Subgoal Provers (2022–2024)

- **Draft-Sketch-Prove (2022)**: informal sketch → formalize → fill subgoals recursively
- **LeanDojo (2023)**: retrieval-augmented premise selection
- **DeepSeek-Prover-V1.5**: large-scale LLM fine-tuned on formal corpora
- **DeepSeek-Prover-V2, Kimina (2024)**: subgoal decomposition; divide-and-conquer

The *subgoal paradigm* becomes dominant: decompose the goal, solve recursively.

NTP: Bootstrapping and Self-Improvement (2024–2025)

- **AlphaProof (2024)**: RL on formal proofs; solved 4/6 IMO 2024 problems
- **Gödel-Prover V2, STP-Prover**: self-play bootstrapping; IMO gold-medal level
- **SeedProver, Hilbert**: automated lemma generation; hundreds of results auto-generated
- **Gauss (2025)**: autoformalization of *Sphere Packing* — an ongoing research project
Cost: enormous compute; not practical for routine library use today

These Systems Train on Formal Corpora

- All NTP systems train on large datasets of formal proofs
- The structure and style of training proofs shapes what the model learns
- Bootstrapping: each generation of provers trains on the previous generation's outputs

The Data Quality Hypothesis

Performance of downstream provers is influenced by training data **quality**, not just quantity.

Correct $\neq$ Good

- Lean's kernel checks *correctness* — not readability, length, or structure
- AlphaProof's proofs: reportedly “arcane, non-human structure”
- Human first-drafts: written to compile, not to be maintained
- AI-generated proofs: optimized for passing the kernel, not for downstream use

There is no feedback mechanism pushing toward high-quality proofs.

The Review Bottleneck is About to Get Much Worse

- Mathlib already receives more PRs than reviewers can comfortably handle
- AI-generated proofs are about to flood formal math libraries
- AI proofs often require *more* reviewer effort: arcane structure, unusual lemma usage
- Manual curation at library scale is not viable

We need automated proof quality improvement.

Two Applications of Proof Optimization

Library Curation

Rewrite incoming proofs to meet Mathlib's style guidelines automatically.

Reduce maintainer burden; raise baseline quality of contributions.

Training Data Quality

Rewrite proof corpora into structured, modular form before training the next NTP generation.

Better structure → better downstream provers.

Both require: given a verified proof, rewrite it to be *better* while staying correct.

Proof Optimization

Proof Triples: Spaces and Notation

Definition (Proof Context, Statement, Proof)

Let $\mathcal{C}$ denote *proof contexts* (imports, open namespaces, local declarations), $\mathcal{X}$ *theorem statements*, and $\mathcal{Y}$ *proofs*.

We work with triples $(c, x, y) \in \mathcal{C} \times \mathcal{X} \times \mathcal{Y}$, where y is a purported proof of x in context c .

Proof Triples: Verifier and Valid Proofs

Definition (Verifier and Valid Proof Set)

A *verifier* is a computable function $v : \mathcal{C} \times \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$. The set of *valid proofs* of x in c is

$$\mathcal{F}(c, x) := \{y \in \mathcal{Y} : v(c, x, y) = 1\}.$$

Two proofs $y, y' \in \mathcal{F}(c, x)$ are *semantically equivalent*: they prove the same fact but may differ in structure. We use Lean 4 as the verifier throughout.

Definition (Optimization Metric)

An *optimization metric* is any computable function

$$\mu : \mathcal{C} \times \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}.$$

Higher values are better.

Any notion of proof quality can be a metric: length, structure, dependency footprint, readability, ...

Definition (Improvement Score)

Given (c, x, y_0) with $v(c, x, y_0) = 1$ and metric μ , the *improvement score* of candidate y is

$$S_\mu(y, y_0 \mid c, x) := [\mu(c, x, y) - \mu(c, x, y_0)] \cdot v(c, x, y).$$

- Zero if y does not compile
- Zero if $\mu(y) \leq \mu(y_0)$ (no metric improvement)
- $S_\mu(y_0, y_0 \mid c, x) = 0$: returning the original is always feasible but trivial
- **Any $\hat{y}$ with $S_\mu > 0$ is a strictly improved proof**

The Optimization Objective

The **proof optimization problem** for a single theorem is

$$\hat{y} = \arg \max_{y \in \mathcal{Y}} S_{\mu}(y, y_0 \mid c, x).$$

- Since $S_{\mu}(y_0, y_0) = 0$, returning the original is always feasible
- Any $\hat{y}$ with $S_{\mu} > 0$ is a *strictly better* proof
- In practice, we approximate via a language model generator G and an augmentation scaffold Ψ :

$$y_1, \dots, y_n \sim G(\cdot \mid \Psi(c, x, y_0), \mu)$$

- Then select via best-of- n

Proof Optimization Subsumes NTP

Remark (Generalization)

Neural theorem proving (NTP) is a special case of proof optimization.

Construction:

- Set $y_0 = \emptyset$ (empty, syntactically invalid proof)
- Set $\mu := \mu_{\text{comp}}(c, x, y) := v(c, x, y) \in \{0, 1\}$
- Then $S_\mu(y, \emptyset \mid c, x) = v(c, x, y)$
- Finding $\hat{y}$ with $S_\mu > 0$ is exactly: *find any valid proof*

Any improvement to the proof optimization framework also improves neural theorem proving.

Best-of- n Selection

Definition (Best-of- n)

Given n candidate proofs $y_1, \dots, y_n$ and original y_0 , the *best-of- n output* is

$$\hat{y} = \arg \max_{0 \leq i \leq n} S_{\mu}(y_i, y_0 \mid c, x).$$

y_0 is always included: $\hat{y}$ is *never worse* than the original proof.

Best-of- n : Properties

- $\mathbb{E}[\max_i S_\mu(y_i, y_0)]$ is non-decreasing in n : more samples = better expected performance
- When all candidates fail to compile, fall back to y_0 (score = 0, no regression)
- Best-of- n defines the outer selection loop in ImProver and ImProver 2

Dataset-Level Evaluation: Three Quantities

Given $\mathcal{D} = \{(c_i, x_i, y_{0,i})\}_{i=1}^N$, best-of- n outputs $\hat{y}_i$:

Definition (Mean Improvement Score (primary))

$$\bar{S}_\mu(\mathcal{D}) := \frac{1}{N} \sum_{i=1}^N S_\mu(\hat{y}_i, y_{0,i} \mid c_i, x_i)$$

Jointly rewards correctness and magnitude. Cannot be inflated by returning y_0 .

Definition (Compilation Accuracy)

$$\mathcal{A}(\mathcal{D}) := \frac{1}{N} \sum_{i=1}^N \mathbf{v}(c_i, x_i, \hat{y}_i)$$

Fraction of theorems with a compiling output.

Definition (Improved Accuracy)

$$\mathcal{A}_\mu^+(\mathcal{D}) := \frac{1}{N} \sum_{i=1}^N \mathbf{1}[S_\mu(\hat{y}_i, y_{0,i}) > 0]$$

Fraction with compiling proof *and* strictly positive improvement.

Datasets

Dataset	Level	N
Mathematics in Lean (MIL)	Undergraduate	72
Compfiles	Competition (IMO/etc.)	26
Mathlib	Research library	43
miniCTX-v2	7 research repos	held-out

- MIL, Compfiles, Mathlib: primary benchmarks for **ImProver**
- miniCTX-v2 (Mathlib, HEPLean, PFR, Carleson, ConNF, FLT, Seymour): primary benchmark for **ImProver 2**; in-context, real project dependencies

Metric: Length

Let $|y|_{\text{tac}}$ denote the number of tactic invocations in proof y .

Definition (Length Metric)

$$\mu_{\text{len}}(c, x, y) := -|y|_{\text{tac}}$$

- Shorter proofs are easier to read and maintain
- Less compilation overhead; proof golfing is common in formal math
- Used in both ImProver and ImProver 2
- **Failure mode:** replace structured proof with opaque one-liner

Metric: Declarativity

Let $|y|_{\text{have}}$ denote the number of explicitly typed `have` tactics in y .

Definition (Declarative Metric)

$$\mu_{\text{decl}}(c, x, y) := \frac{|y|_{\text{have}}}{|y|_{\text{tac}}}$$

- Declarative proofs make logical structure explicit: each `have` names and proves an intermediate claim
- More readable, modular, and maintainable
- Used in ImProver
- **Failure mode:** `have`-padding (nominally typed stubs, no deductive role); addressed by modularity in ImProver 2

Metrics: Mixed and Completion

Definition (Mixed Metric)

$$\mu_{\text{mix}}(c, x, y) := 5 \cdot |y|_{\text{have}} - |y|_{\text{tac}}$$

Each have tactic “pays for” four additional tactics. Jointly optimizes brevity and structure.

Definition (Completion Metric)

$$\mu_{\text{comp}}(c, x, y) := v(c, x, y) \in \{0, 1\}$$

Any correct proof achieves the optimum. Used in ImProver only to demonstrate NTP generalization.

Both metrics have failure modes (see Degenerate Solutions slide).
Mixed partially mitigates length+declarativity failures simultaneously.

Metric: Dependencies

Let $\text{Deps}(c, x, y)$ be the set of external theorem/lemma names explicitly referenced in y .

Definition (Dependency Metric)

$$\mu_{\text{dep}}(c, x, y) := - |\text{Deps}(c, x, y)|$$

Note: measures the explicit *name* footprint, not logical dependency. `rw [a, b, c] → omega` reduces the count even if the logical dependency is unchanged.

Metric: Dependencies — Rationale

- Minimizes explicit dependency footprint (fewer named lemma references)
- Encourages self-contained proofs: less brittle across library versions
- Consistent with Mathlib style guidance (prefer automation over long rewrite chains)
- *Not* a degenerate metric: `omega/simp` replacing explicit rewrites is exactly the desired behavior
- Introduced in ImProver 2; evaluated on miniCTX-v2

Metric: Modularity — Lean Elaboration Semantics

Goal: count *independent, non-trivial* subproof obligations — with kernel-level guards against reward hacking.

Lean elaborates tactic proofs via a *metavariable context*

$M = (\Delta, \sigma)$:

- Δ : finite set of metavariable declarations $?m : (\Gamma_{?m} \vdash A_{?m})$
- σ : partial assignment mapping metavariables to terms
- A **goal** is an unassigned metavariable $?m \in \Delta \setminus \text{dom}(\sigma)$

At each step i , tactic τ_i focuses goal $?m_i$, assigns $\sigma_{i+1}(?m_i) = t_i$, and introduces new metavariables (subgoals).

Definition (Direct Children)

$\text{Children}(\tau_i) := \{\text{metavariables free in } t_i\}$ — the direct subgoals of the focused goal.

Metric: Modularity — Spawned Goals

Not all new obligations are direct children. Tactics like `have/calc` introduce *independent* sub-obligations.

Let $\text{Current}_i := \{?m_i\} \cup \text{Children}(\tau_i)$ and
 $S_{i+1} := S_i \cup \text{Children}(\tau_i)$.

Definition (Spawned Goals)

$$\text{Spawned}(\tau_i) := (\text{Current}_i \setminus \text{Children}(\tau_i)) \setminus S_i$$

Goals that first appear at step i , are not direct logical subgoals, and correspond to independent sub-obligations introduced by `have/calc`.

The *proof graph* $\mathcal{T}(y)$ records normal edges ($i \rightarrow j$: direct subgoal) and spawned edges ($i \rightsquigarrow j$: new sub-obligation). Extracted from Lean's `InfoTree`.

Metric: Modularity — Guards Against Reward Hacking

Spawned goals must pass four guard conditions (all at the **kernel level**):

1. **Non-trivial**: subtree has > 2 tactic steps (excludes goals solved by `simp/linarith`)
2. **No duplicates**: sequent variant hash $\neq$ any previously seen goal (up to $\alpha/\beta\delta\iota$ -normalization)
3. **No wrappers**: target is not a definitional wrapper of the parent goal
4. **Effectiveness**: introduced hypotheses are used in the outer proof or another effective subtree

Guards use hash-based canonical representations: base target hash + sorted hypothesis hashes + sequent variant hashes.

Metric: Modularity — Effective Spawned Goals

Definition (Effective Spawned Goals)

Define the monotone operator Φ on sets of spawned roots:

$$\Phi(S) := \{ g \mid \text{Intro}(g) \text{ used outside all spawned subtrees, or} \\ \exists g' \in S, g' \neq g, \text{Intro}(g) \text{ used in subtree of } g' \}$$

The set of *effective spawned goals* $\mathcal{E}(y)$ is the **least fixed point** of Φ , restricted to goals satisfying all four guard conditions.

Guaranteed to terminate: only finitely many spawned roots.

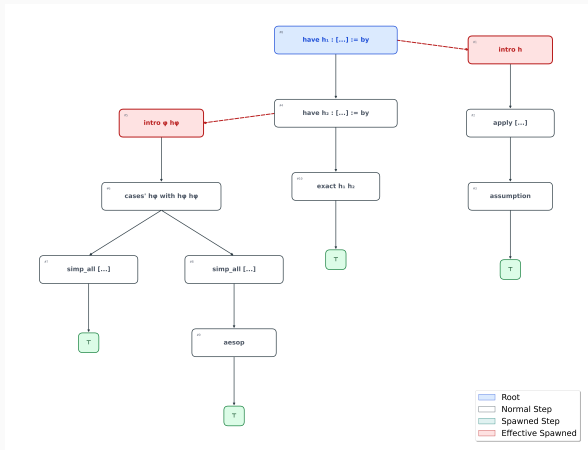
Metric: Modularity — The Metric

Definition (Modularity Metric)

$$\mu_{\text{mod}}(c, x, y) := |\mathcal{E}(y)|$$

- Counts the number of effective spawned goals satisfying all four guard conditions
- Guards operate at the **term level**, not tactic syntax — substantially narrowing the reward-hacking surface relative to the declarative metric
- Kernel-level evaluation: immune to syntactic reward hacking

Modularity: Proof Tree Example



Dashed red arrows = spawned edges (`have/calc`). Red nodes = effective spawned goals ($|\mathcal{E}| = 2$ here). Normal edges = direct subgoals.

Metrics Summary

Metric	Definition	System	Guard level
Length	$- y _{\text{tac}}$	ImProver, IP2	Syntactic
Declarative	$ y _{\text{have}}/ y _{\text{tac}}$	ImProver	Syntactic
Mixed	$5 y _{\text{have}} - y _{\text{tac}}$	ImProver	Syntactic
Completion	$v(c, x, y)$	ImProver	None
Dependencies	$- \text{Deps}(c, x, y) $	ImProver 2	Semantic
Modularity	$ \mathcal{E}(y) $	ImProver 2	Kernel-level

ImProver metrics: syntactic proxies, easy to compute.

ImProver 2 metrics: use Lean elaboration semantics with explicit reward-hacking guards.

Degenerate Solutions: ImProver Metrics

Every metric has characteristic failure modes. The choice of μ is the user's responsibility.

- **Length:** replace structured proof with opaque single-line proof term — shorter, but obscures structure
- **Declarativity:** insert have-padding — nominally typed stubs with no deductive role
- **Mixed:** inherits both; each metric lessens the other's failure mode impact
- **Completion:** any correct proof is optimal (by design: used only for NTP generalization demo)

ImProver 2 metrics use kernel-level guards to explicitly block common degenerate patterns.

- **Dependencies:** $\text{rw } [a,b,c] \rightarrow \text{omega}$ reduces count but this is *not* degenerate — aligned with Mathlib automation style
- **Modularity:** duplicate haves, single-automation spawned goals, wrapper subgoals — all **explicitly blocked** by kernel-level guards (term-level, not syntactic)

ImProver

The Core Challenge

- Lean proofs are extremely sensitive: most perturbations are invalid
- A model must understand the math *and* the target metric simultaneously
- **Without proof-state context:** the model cannot see intermediate goals/hypotheses
- **Without error feedback:** no signal when generation fails to compile

A single prompt-and-sample loop is not sufficient for reliable proof optimization.

ImProver: An Agentic Scaffold

ImProver wraps GPT-4o in a scaffold Ψ_{Imp} . Given (c, x, y_0) and metric μ :

$$y_1, \dots, y_n \sim G(\cdot \mid \Psi_{\text{Imp}}(c, x, y_0), \mu), \quad \hat{y} = \text{best-of-}n$$

If no candidate improves on y_0 , ImProver returns y_0 — 100% compilation guaranteed.

The scaffold Ψ_{Imp} augments generation with four components:

1. **Chain-of-States (CoS)**: annotate y_0 with intermediate Lean proof states
2. **Output formatting**: structured constraints on syntactic form of output
3. **Sampling and refinement**: past generation history and evaluations
4. **RAG**: relevant examples and docs from Mathlib and TPiL

Chain-of-States: Intuition

- Each Lean tactic transforms the *proof state*: hypotheses and remaining goals
- This information is invisible in the raw proof text
- It *is* available from Lean's InfoTree during compilation
- **CoS**: extract these states symbolically and insert them as comments before each tactic

Analogous to chain-of-thought prompting: CoS makes intermediate *symbolic states* explicit, not just reasoning steps.

Chain-of-States: Formal Definition

Definition (Chain-of-States)

Given proof y_0 , from InfoTree extract
(goalsBefore $_i$, τ_i , goalsAfter $_i$) for each step i .

$$\Psi_{\text{cos}}(y_0) = \langle (\text{goalsBefore}_i, \tau_i, \text{goalsAfter}_i) \rangle_{i=1}^T,$$

serialized as comments adjacent to each τ_i in the proof text.

- States extracted from Lean's elaboration stages (symbolic, not approximate)
- Proof data stored in `Lean.Elab.InfoTree`
- Hidden kernel states become explicit cues the model can condition on

Chain-of-States: Before and After

Without CoS

example :

```
s ∩ t ∪ s ∩ u
⊆ s ∩ (t ∪ u) := by
rintro x
  (⟨xs,xt⟩ | ⟨xs,xu⟩)
. use xs; left; exact xt
. use xs; right; exact xu
```

With CoS

example :

```
s ∩ t ∪ s ∩ u
⊆ s ∩ (t ∪ u) := by
rintro x
  (⟨xs,xt⟩ | ⟨xs,xu⟩)
/- case inl.intro
  xs : x ∈ s, xt : x ∈ t
  ⊢ x ∈ s ∩ (t ∪ u) -/
. use xs; left; exact xt
/- Goals Solved! -/
. use xs; right; exact xu
```

The ablation study confirms CoS is the **single largest contributor**: enabling CoS nearly doubles the improvement score for declarativity and substantially improves accuracy across all metrics.

Output Formatting

LLM outputs often contain ancillary and syntactically invalid content. Three output formats tested:

1. `string`: raw proof text (default baseline)
2. `string list`: model outputs tactic sequence as list of strings, one tactic per element
Isolates individual steps; makes parsing and validation straightforward
3. `string tree`: model outputs proof as a tree of strings
Encourages explicit reasoning about hierarchical proof structure

Ablation Winner

`string list` with CoS enabled achieves the best mean improvement score.

Error Correction (inspired by self-debugging in code generation):

- n refinement iterations; each receives: input, output, metric score, correctness, error messages from the last `prev_num` iterations
- Model observes its own mistakes and attempts targeted corrections
- Error messages forwarded from Lean's elaborator (syntax and type errors)

Compound Sampling Methods

Compound methods nest best-of- n and refinement (mn total LLM calls):

- `best_of_n(refinement( $m$ ),  $n$ )`: best-of- n where each call is an m -step refinement
- `refinement(best_of_n( $m$ ),  $n$ )`: n -step refinement, each iteration runs best-of- m

Ablation Winner

`refinement(best_of_3, 5)`: 5-step refinement, best-of-3 per step (= 15 LLM calls). Optimal balance of correction and sampling diversity.

RAG: Example Retrieval

ImProver uses **Maximum Marginal Relevance (MMR)** retrieval to select relevant context.

- For user-specified k , retrieve the top- k most relevant human-written proof rewrites for the specific metric being optimized
- Provided as multi-shot demonstrations in the prompt
- Helps the model understand the metric's intent concretely; mitigates the risk of degenerate solutions (see §Degenerate Solutions)
- Ablation winner: $k = 10$ examples

Document retrieval from two fixed vector databases:

- **Mathlib**: top- k documents with highest MMR score against the current theorem statement — finds relevant lemmas and tactics
- **TPiL** (*Theorem Proving in Lean*): top- k passages scored against the theorem *and all current error messages* — helps the model find syntax guides and fix its own compilation errors

ImProver Experiments

Experimental Setup

Datasets

Dataset	N
MIL (undergrad)	72
Compfiles (competition)	26
Mathlib (research)	43

Metrics: length, declarative, mixed

Generator: GPT-4o
(gpt-4o-2024-08-06)

Baselines

GPT-4o without scaffold

GPT-4o-mini without scaffold

Ablation protocol

Factorial testing on MIL subset;
hold best config, vary next group.

Primary metric: $\bar{S}_\mu$

Ablation Study: Six Groups

Sequential factorial design on MIL subset (length metric fixed):

1. **Model:** GPT-4o-mini vs. GPT-4o; string output, no features
2. **Output format and CoS:** all format types $\times$ CoS on/off; GPT-4o fixed
3. **Example retrieval:** $k \in \{0, 3, 5, 7, 10\}$
4. **Sampling method:** best-of- n vs. refinement; forwarding strategies
5. **n and model:** $n \in \{3, 5, 7, 10, 15\}$ for GPT-4o; $n = 20$ for mini
6. **Compound methods and RAG:** compound strategies $\pm$ document retrieval

Each group: select parameters maximizing $\bar{S}_\mu$; hold fixed for subsequent groups.

Ablation Results

	$\bar{S}_\mu$	$\mathcal{A}$	$\mathcal{A}_\mu^+$
GPT-4o-mini	0	3.62%	0%
GPT-4o	7.03	35.77%	15.33%
+ Output and CoS	8.04 / 6.31	64.96% / 44.53%	21.17% / 16.06%
+ Example Retrieval	9.34 / 5.67	63.5% / 67.15%	21.9% / 16.79%
+ Sampling Method	15.35 / 9.34	83.21% / 63.5%	36.5% / 21.9%
+ n and Model	23.51 / 3.65	89.47% / 78.95%	45.61% / 8.77%
+ Combos and RAG	34.88 / 28.25	60.61% / 84.38%	54.55% / 53.12%
ImProver	34.88	100%	54.55%

best / worst within group. Each component adds incremental improvement.

Ablation: CoS on Declarativity

CoS Declarativity Ablation (CoS nearly doubles score):

	$\bar{S}_\mu$	$\mathcal{A}$	$\mathcal{A}_\mu^+$
GPT-4o	4.97	37.5%	12.5%
ImProver, CoS Disabled	9.23	100.0%	28.12%
ImProver	16.69	100.0%	46.88%

CoS Ablation: Why It Works

	$\bar{s}_{\mu}$	$\mathcal{A}_{\mu}^{+}$
Without CoS	9.23	28.12%
With CoS	16.69	46.88%

CoS exposes local hypotheses and goals that declarative have tactics name explicitly.

Enabling CoS nearly doubles the improvement score on declarativity.

Syntax Guidance Ablation (without error message forwarding):

	$\bar{S}_\mu$	$\mathcal{A}$	$\mathcal{A}_\mu^+$
GPT-4o	11.00	42.42%	21.21%
ImProver, No Syntax Guidance	23.42	100.0%	46.88%
ImProver	28.94	100.0%	59.38%

Primary drivers: CoS, example retrieval, RAG. Syntax guidance improves $\mathcal{A}^+$ by $\approx 13\%$.

Main Results: Average Across Datasets

Metric	Model	$\bar{S}_\mu$	$\mathcal{A}$	$\mathcal{A}_\mu^+$
Length	GPT-4o	3.7	26.36%	8.31%
	ImProver	20.96	100.0%	35.44%
Declarativity	GPT-4o	2.21	18.75%	6.13%
	ImProver	9.34	100.0%	24.56%
Mixed	GPT-4o	3.51	14.70%	5.11%
	ImProver	27.31	100.0%	30.55%

ImProver outperforms GPT-4o by **566%** on length, **423%** on declarativity, **778%** on mixed.

ImProver guarantees 100% compilation via fallback to y_0 .

Main Results: MIL

Metric	Model	$\bar{S}_\mu$	$\mathcal{A}$	$\mathcal{A}_\mu^+$
Length	GPT-4o	6.25	37.5%	14.42%
	ImProver	30.54	100.0%	50.0%
Declarativity	GPT-4o	4.18	28.85%	11.54%
	ImProver	13.45	100.0%	34.21%
Mixed	GPT-4o	3.70	13.51%	0.0%
	ImProver	43.55	100.0%	45.94%

Main Results: Compfiles and Mathlib

Compfiles ($N = 26$)

	$\bar{S}$	$\mathcal{A}$	$\mathcal{A}^+$
<i>Length</i>			
GPT-4o	2.75	11.5%	5.1%
ImProver	18.86	100%	34.6%
<i>Declarativity</i>			
GPT-4o	0.39	14.1%	1.3%
ImProver	5.74	100%	19.2%
<i>Mixed</i>			
GPT-4o	3.96	26.9%	20.0%
ImProver	20.60	100%	23.1%

Mathlib ($N = 43$)

	$\bar{S}$	$\mathcal{A}$	$\mathcal{A}^+$
<i>Length</i>			
GPT-4o	0.0	16.7%	0.0%
ImProver	6.19	100%	11.5%
<i>Declarativity</i>			
GPT-4o	0.0	4.7%	0.0%
ImProver	4.63	100%	11.6%
<i>Mixed</i>			
GPT-4o	2.92	9.3%	4.7%
ImProver	4.16	100%	9.3%

Improvement score decreases with dataset difficulty; ImProver maintains 100% compilation throughout.

Neural Theorem Proving Evaluation

Using the completion metric (μ_{comp}) with $y_0 = \emptyset$ (empty proof)
— demonstrating that proof optimization *subsumes* NTP:

	MIL-C04 Pass@15	MIL-C08 Pass@15	MIL Pass@15	MiniF2F-test Pass@8
GPT-4o	18.18%	25.00%	21.73%	9.02%
ImProver	45.45%	33.33%	39.13%	16.39%
Lean Expert Iter.	—	—	—	34.5%

Specialized NTP systems (trained for NTP) lead on MiniF2F; ImProver achieves competitive performance without NTP-specific training.

Qualitative: Length Optimization

MIL group theory (map composition): $9 \rightarrow 7$ tactics

Original

```
example (ϕ:G →* H)(ψ:H →* K)
  (S:Subgroup G):
  map (ψ.comp ϕ) S =
  map ψ (S.map ϕ) := by
ext x; simp only [mem_map]
constructor
. rintro ⟨y,y_in,hy⟩
  exact ⟨ϕ y,⟨y,y_in,rfl⟩,hy⟩
. rintro ⟨y,⟨z,z_in,hz⟩,hy⟩
  use z, z_in
  calc ψ.comp ϕ z
    = ψ(ϕ z) := rfl
  _ = ψ y := by congr
```

ImProver (length-optimized)

```
example (ϕ:G →* H)(ψ:H →* K)
  (S:Subgroup G):
  map (ψ.comp ϕ) S =
  map ψ (S.map ϕ) := by
ext x; simp only [mem_map]
constructor
. rintro ⟨y,y_in,hy⟩
  exact ⟨ϕ y,⟨y,y_in,rfl⟩,hy⟩
. rintro ⟨y,⟨z,z_in,hz⟩,hy⟩
  exact ⟨z,z_in,
    (congr_arg ψ hz).trans hy⟩
```

calc block $\rightarrow$ one proof term.

Qualitative: Declarativity Optimization

Compfiles (IMO 2019 P1): additive_to_int_linear

Original (opaque rewrite chain)

```
lemma additive_to_int_linear
  (f :  $\mathbb{Z} \rightarrow \mathbb{Z}$ )
  (h:  $\forall (x\ y : \mathbb{Z}),$ 
    f (x + y) = f x + f y):
   $\exists\ c, \forall\ a, f\ a = c * a :=$  by
  let g := AddMonoidHom
    .toIntLinearMap
    <| AddMonoidHom.mk' f h
  refine (f 1, fun a => ?_)
  change g a = g 1 * a
  rw [mul_comm,
    ← smul_eq_mul,
    ← LinearMap.map_smul,
    smul_eq_mul, mul_one]
```

ImProver (named intermediates)

```
lemma additive_to_int_linear
  (f :  $\mathbb{Z} \rightarrow \mathbb{Z}$ )
  (h:  $\forall (x\ y : \mathbb{Z}),$ 
    f (x + y) = f x + f y):
   $\exists\ c, \forall\ a, f\ a = c * a :=$  by
  let g := ...
  have linear_property :
     $\forall\ a, f\ a = g\ a :=$  by
    intro a; rfl
  have g_smul :
     $\forall\ a, g\ a = g\ 1 * a :=$  by
    intro a
  rw [mul_comm, ← smul_eq_mul,
    ← LinearMap.map_smul,
    smul_eq_mul, mul_one]
  exact (f 1, fun a =>
    (linear_property a).trans
    (g_smul a))
```

Qualitative: Mixed Optimization

MIL group theory: eq_bot_iff_card (7 tactics $\rightarrow$ 3)

Original

```
lemma eq_bot_iff_card
  {H : Subgroup G} [Fintype H] :
  H =  $\perp$   $\leftrightarrow$  card H = 1 := by
  suffices
    ( $\forall x \in H, x = 1$ )  $\leftrightarrow$ 
    ( $\exists x \in H, \forall a \in H, a = x$ )
  by simpa [eq_bot_iff_forall,
    card_eq_one_iff]
  constructor
  . intro h; use 1, H.one_mem
  . rintro  $\langle y, -, hy' \rangle x hx$ 
    calc x = y := hy' x hx
      = 1 :=
      (hy' 1 H.one_mem).symm
```

ImProver (shorter + declarative)

```
lemma eq_bot_iff_card
  {H : Subgroup G} [Fintype H] :
  H =  $\perp$   $\leftrightarrow$  card H = 1 := by
  have key :
    ( $\forall x \in H, x = 1$ )  $\leftrightarrow$ 
    ( $\exists x \in H,$ 
       $\forall a \in H, a = x$ ) :=
     $\langle \lambda h \Rightarrow \langle 1, H.one\_mem, h \rangle,$ 
       $\lambda \langle y, -, hy' \rangle x hx \Rightarrow$ 
        hy' x hx  $\rangle$ 
  simpa [eq_bot_iff_forall,
    card_eq_one_iff] using key
```

Both shorter *and* more declarative.

ImProver: Limitations

1. **Cost:** many GPT-4o API calls + document retrieval per theorem; library-scale deployment infeasible at current pricing
2. **Closed-source:** tightly coupled to GPT-4o; non-reproducible; subject to model deprecation
3. **Small evaluation:** only $72 + 26 + 43 = 141$ theorems total; toy datasets not representative of real research repositories with inter-file dependencies
4. **Weak metrics:** declarativity gameable via have-padding; no kernel-level structural or maintainability metric

These limitations motivate **ImProver 2**.

ImProver 2

ImProver 2: Addressing the Limitations

ImProver problems

- ✗ Expensive frontier model (GPT-4o)
- ✗ Closed-source, non-reproducible
- ✗ Toy evaluation datasets
- ✗ Gameable syntactic metrics

ImProver 2 solutions

- ✓ 7B open-weight model (DS-R1-Distill)
- ✓ Fully reproducible; locally hosted
- ✓ miniCTX-v2: 7 real research repos
- ✓ Kernel-level structural metrics

Three Core Components

Neurosymbolic scaffold Ψ + **IRPO training** + **Replay buffer**

The ImProver 2 Core Loop

Given base model G_0 , at iteration t :

1. **Generate**: run $G_t + \text{scaffold } \Psi$ on training set; n candidates per theorem $\rightarrow \mathcal{D}_{\text{nr}}^{(t)}$
2. **Replay**: interleave $\mathcal{D}_{\text{nr}}^{(t)}$ with filtered prior-round data $\mathcal{D}_{\text{re}}^{(t-1)}$
 $\rightarrow \mathcal{D}_{\text{re}}^{(t)}$
3. **Filter**: remove too-easy problems; partition winners/losers
 $\rightarrow \mathcal{D}_{\text{fil}}^{(t)}$
4. **Pair**: construct preference pairs $\rightarrow \mathcal{D}_{\text{IRPO}}^{(t)}$
5. **Train**: one IRPO epoch on $\mathcal{D}_{\text{IRPO}}^{(t)}$ $\rightarrow G_{t+1}$
6. **Evaluate**: test G_{t+1} on held-out miniCTX-v2 (best@16)

Repeat until validation improvement regresses or compute budget exhausted.

Neurosymbolic Scaffold: Three Channels

The scaffold Ψ augments every generation prompt with three views of the formal proof environment:

$$\Psi : I \mapsto (I, \Psi_{\text{ctx}}(I), \Psi_{\text{cos}}(I), \Psi_{\text{inf}}(I))$$

1. **Context slice** Ψ_{ctx} : compact formal context (declarations reachable from the theorem)
2. **Chain-of-States** Ψ_{cos} : goal-state traces from Lean's InfoTree
3. **Auto-informalization** Ψ_{inf} : natural language translation of the proof

Each channel adds a different *kind* of information. Generation is still prompted to be formal; correctness still judged formally.

Channel 1: Context Slice

When proofs rely on many lemmas and definitions, the context is large. Ψ_{ctx} extracts a minimal, deterministic bundle.

- **CST view** (textual reachability): harvest all surface identifiers in x and y_0 with source spans
- **AST view** (semantic reachability): resolve identifiers to fully-qualified symbols; record declaration categories and provenance

Context slice: reachable subgraph of the entity graph:

$$S(c, x, y_0) = \text{Reach}(G_{\text{ent}}, \text{touch}(x, y_0))$$

Serialized with declaration type and source snippet.

Invariant to formatting; robust to α -renaming. Slice size grows with reachable subgraph, not raw file size.

Channel 2: Chain-of-States

- Reuses the Ψ_{cos} transformation from ImProver (same InfoTree extraction)
- Exposes intermediate proof states of y_0 to the model as comments before each tactic
- Provides a richer signal than the raw proof text alone

Tells the model *what each tactic does*: the exact symbolic effect on the proof state at every step.

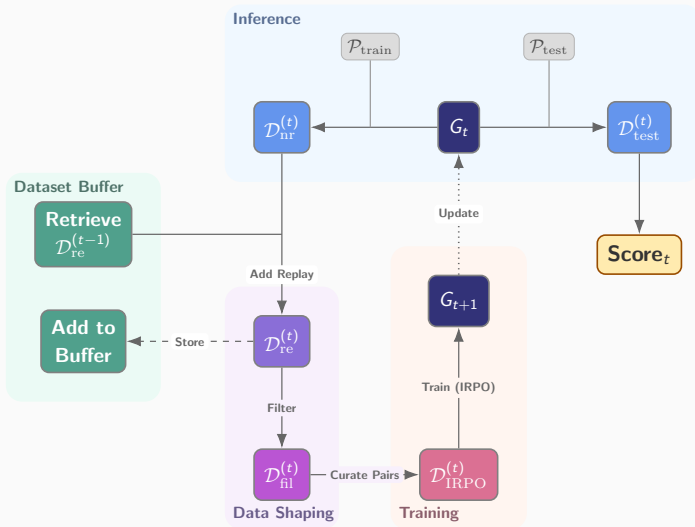
Both ImProver and ImProver 2 use the same CoS extraction. In ImProver 2 it is one of three scaffold channels; in ImProver it was the primary context source.

Channel 3: Auto-Informalization

- Prompt a language model with the CoS-annotated proof to translate it into natural language
- Explains the effect of each tactic on the proof state in prose
- Provides a fuzzy, high-level description robust to syntactic variation
- Inspired by the Draft-Sketch-Prove paradigm: natural language sketches improve LLM formal generation

Tells the model *what the proof is about* at a human-readable mathematical level, while generation and evaluation remain fully formal.

Training Pipeline: Flowchart



Replay Buffer: Improvement Rate

Motivation: indiscriminate self-training leads to model collapse.
The replay buffer mixes new and historical data.

For problem T , the *improvement rate* at iteration j :

$$\pi_T^{(j)} = \frac{1}{n} \sum_{i=1}^n \mathbb{I} \left[v(c_T, x_T, y_{T,i}^{(j)}) = 1 \wedge \hat{s}_{T,i}^{(j)} > 0 \right]$$

A problem is **replay-eligible** if it had at least one improved compiling solution in some prior iteration.

Replay Buffer: Construction

Buffer construction (parameterized by replay proportion ρ , mode, cap $\pi_{\max}$, gap γ):

1. **Mark:** tag problems as REPLAY or FRONTIER to achieve target fraction ρ ; downsample easiest replay items
($\pi_T > \pi_{\max}$)
2. **Merge:** for replay items, splice candidate set from prior round via join/replace/mark modes

Replay Buffer: Filtering and Winner/Loser Partition

Filtering: $\tilde{\mathcal{P}}^{(t)} = \mathcal{P}_{\text{train}} \setminus \{T \mid \pi_T^{(t)} > \pi_{\max}\}$

Winner/Loser partition ($\delta^{(t)} = \gamma$ -th percentile of scores):

- $W_T^{(t)} = \{y \in \tilde{\mathcal{Y}}_T^{(t)} : v(\cdot) = 1 \wedge \hat{s}_{T,y} > \delta^{(t)}\}$
- $L_T^{(t)} = \tilde{\mathcal{Y}}_T^{(t)} \setminus W_T^{(t)}$

Preference pairs (two families):

$$\mathfrak{P}_T^{\ell \rightarrow w} = \{(b, g) : b \in L_T^{(t)}, g \in W_T^{(t)}\}$$

$$\mathfrak{P}_T^{w \rightarrow w} = \{(g', g) : g, g' \in W_T^{(t)}, \hat{s}_{T,g} > \hat{s}_{T,g'}\}$$

$\mathfrak{P}^{w \rightarrow w}$ is **novel**: dense reward signal beyond binary correctness (winners ranked by metric score).

IRPO: Loss Formula

IRPO (Iterative Reasoning Preference Optimization): DPO + NLL stabilizer.

$$\begin{aligned}\mathcal{L}_{\text{IRPO}}(T) = & \mathcal{L}_{\text{DPO}}(y_{T,\ell}, y_{T,w} \mid \Psi(c_T, x_T, y_{T,0}), \mu) \\ & + \alpha \mathcal{L}_{\text{NLL}}(y_{T,\ell}, y_{T,w} \mid \Psi(\cdot), \mu)\end{aligned}$$

- **DPO term:** increase likelihood of preferred y_w relative to y_ℓ (contrastive)
- **NLL term:** stabilize training; prevent winner likelihood from degrading across iterations

IRPO: Masking and Training Schedule

- **Masking:** reasoning trace masked; train only on the final `<answer>` block
- **Schedule:** one IRPO epoch per training round on $\mathcal{D}_{\text{IRPO}}^{(t)}$
- **Initialization:** G_{t+1} initialized from G_t (iterative refinement of weights)

Training on masked answer blocks prevents the model from memorizing reasoning chains while still learning the proof structure.

ImProver 2 Experiments

Experimental Setup

Test dataset: miniCTX-v2

- 7 research-level Lean repos
- Mathlib, HEPLean, PFR, Carleson
- ConNF, FLT, Seymour
- In-context: real project imports

Evaluation: best@16 sampling

Primary: $\overline{S}_\mu(\mathcal{D}_{\text{test}})$

Metrics: Length, Dep., Mod.

Base model

DeepSeek-R1-Distill-Qwen-7B

Baselines

DS-R1 family (7B / 14B / 671B)

GPT-4o, GPT-5 variants

GPT-oss-120B

ImProver (prior work)

All baselines evaluated with and without scaffold Ψ to isolate augmentation from training.

Inference Cost Comparison

Model	Type	Cost (Input / Output per 1M tokens)
DeepSeek-R1-7B	Open-weight	Locally hosted
DeepSeek-R1-14B	Open-weight	Locally hosted
DeepSeek-R1-671B	Open-weight	\$0.70 / \$2.50
GPT-4o	API	\$2.50 / \$10
GPT-5-nano	API	\$0.05 / \$0.40
GPT-5-mini	API	\$0.25 / \$2
GPT-oss-120B	Open-weight	Locally hosted
GPT-5-chat	API	\$1.25 / \$10
GPT-5-high	API	\$1.25 / \$10

ImProver 2 (locally hosted 7B) is orders of magnitude cheaper to deploy than frontier API models.

Main Results: Intra-Family (DeepSeek)

Model	Length	Mod.	Dep.
DeepSeek-R1 7B	0.118	0.003	0.050
DeepSeek-R1 14B	0.140	0.037	0.093
DeepSeek-R1 671B	0.308	0.055	0.153
ImProver 2	0.330	0.143	0.206

7B trained model beats 671B base model on all three metrics.

Within the DeepSeek family, larger models score higher (scale helps).
But specialization via IRPO on a 7B model surpasses raw scale.

Main Results: Frontier Models

Model	Length	Mod.	Dep.
GPT-4o	0.336	0.034	0.050
GPT-oss-120B	0.321	0.075	0.181
GPT-5-nano	0.087	0.065	0.106
GPT-5-mini	0.330	0.109	0.203
GPT-5-chat	0.346	0.118	0.046
GPT-5-high	0.660	0.120	0.208
ImProver	0.355	0.088	0.047
ImProver 2	0.330	0.143	0.206

Leads all models on modularity. Essentially tied with GPT-5-high on dependencies (0.206 vs 0.208). Trails GPT-5-high on length (0.330 vs 0.660) — a qualitatively different operating point in inference cost.

Key Finding

Proof optimization is a **specialization** problem as much as a scale problem.

- Within DeepSeek family: larger models score higher (scale helps for length and dependency)
- But ImProver 2 (7B) outperforms DeepSeek 671B on *all three* metrics
- Gap especially pronounced for modularity (0.143 vs 0.055) and dependency (0.206 vs 0.153)

Scale vs. Specialization (cont.)

- Interpretation: structural refactoring requires task-specific adaptation, not just raw reasoning capacity
- ImProver 2 beats GPT-oss-120B (120B parameters) on all three metrics despite starting from a 7B base
- Training on the *right objective* with the right data matters more than scale alone

A 7B model trained for proof optimization > a 671B base model on structural metrics.

Improvement vs. Accuracy: DeepSeek Family

Training raises $\mathcal{A}_\mu^+$ faster than $\mathcal{A}$: riskier edits yield real gains but also more failures.

Model	Length $\mathcal{A}^+/\mathcal{A}$	Mod. $\mathcal{A}^+/\mathcal{A}$	Dep. $\mathcal{A}^+/\mathcal{A}$
DS-R1 7B	0.062 / 0.617	0.003 / 0.570	0.037 / 0.754
DS-R1 14B	0.075 / 0.660	0.034 / 0.775	0.065 / 0.567
DS-R1 671B	0.162 / 0.536	0.048 / 0.536	0.109 / 0.480
ImProver 2	0.131 / 0.657	0.065 / 0.579	0.069 / 0.368

DS-R1 7B (dep.): $\mathcal{A} = 0.754$ but $\mathcal{A}^+ = 0.037$ — compiles but rarely reduces dependencies. After IRPO training: $\mathcal{A}^+$ rises sharply, $\mathcal{A}$ drops moderately.

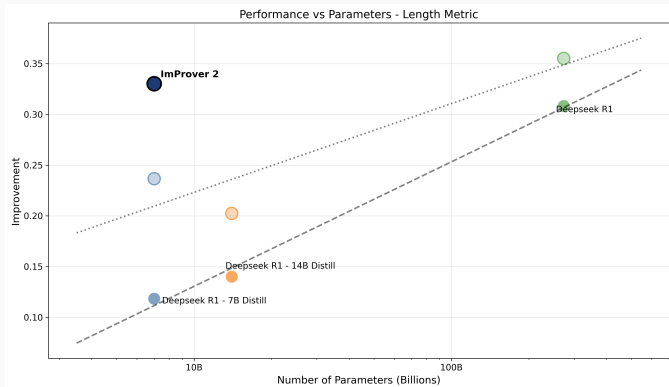
Improvement vs. Accuracy: Frontier Models

All models evaluated with scaffold Ψ ; $\mathcal{A}^+/\mathcal{A}$ format.

Model	Length $\mathcal{A}^+/\mathcal{A}$	Mod. $\mathcal{A}^+/\mathcal{A}$	Dep. $\mathcal{A}^+/\mathcal{A}$
ImProver	0.171 / 0.560	0.068 / 0.597	0.031 / 0.249
GPT-4o	0.158 / 0.595	0.031 / 0.464	0.028 / 0.227
GPT-5-nano	0.044 / 0.461	0.053 / 0.757	0.065 / 0.894
GPT-5-mini	0.159 / 0.623	0.085 / 0.525	0.111 / 0.834
GPT-5-chat	0.165 / 0.586	0.084 / 0.455	0.025 / 0.185
GPT-5-high	0.264 / 0.757	0.092 / 0.656	0.094 / 0.753

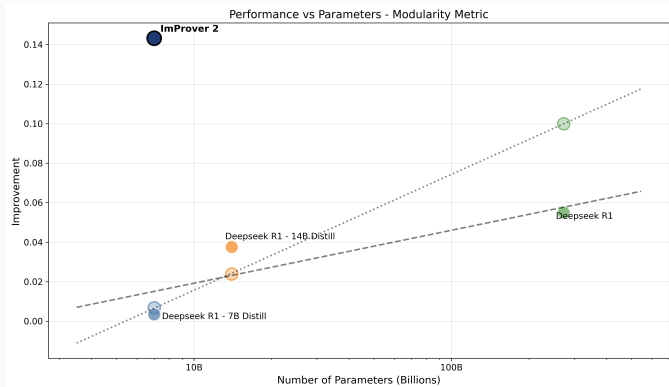
GPT-5-nano: high $\mathcal{A}$ on dep. (0.894) but low $\mathcal{A}^+$ (0.065) — compiles often, improves rarely. Training (ImProver 2) flips this balance.

Performance vs. Parameter Count: Length



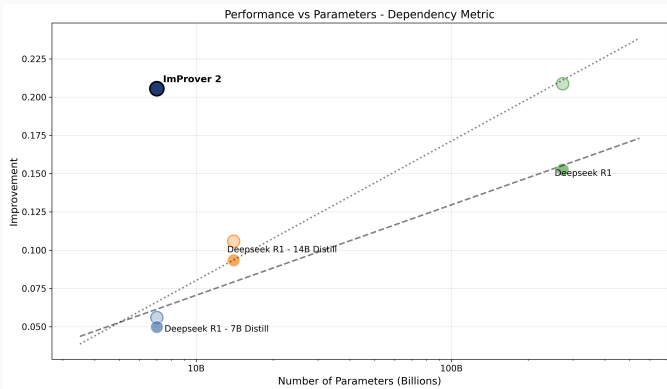
Within DeepSeek family: larger model scores higher. **ImProver 2 (7B)** outperforms DeepSeek-671B.

Performance vs. Parameter Count: Modularity



ImProver 2 leads all models on modularity including 671B and GPT-5-high.

Performance vs. Parameter Count: Dependencies



ImProver 2 (7B) outperforms DeepSeek-671B on all three metrics.
Specialization beats scale.

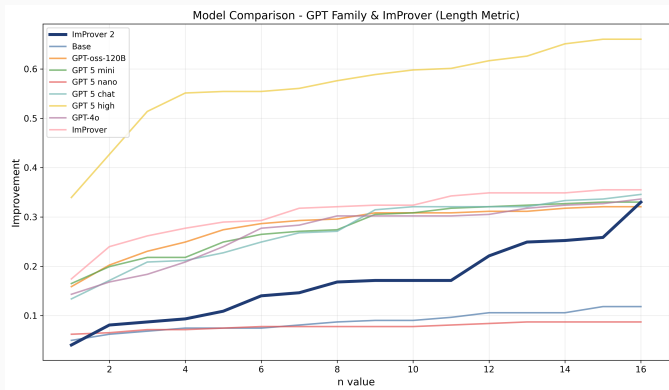
Full Accuracy and Compilation Breakdown

($\mathcal{A}_\mu^+/\mathcal{A}$: improved accuracy / compilation accuracy)

Model	Length	Mod.	Dep.
DS-R1 7B	0.062/0.617	0.003/0.570	0.037/0.754
DS-R1 671B	0.162/0.536	0.048/0.536	0.109/0.480
ImProver 2	0.131/0.657	0.065/0.579	0.069/0.368
GPT-5-high	0.264/0.757	0.092/0.656	0.094/0.753

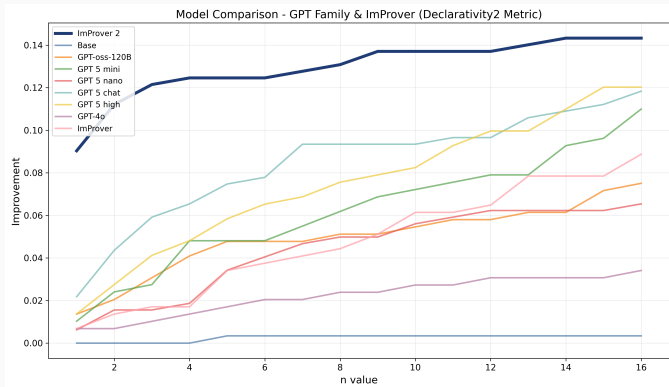
ImProver 2 attempts riskier structural transformations; its $\mathcal{A}^+$ is higher relative to its $\mathcal{A}$ than frontier models.

Frontier Comparison: Length (Best@ n)



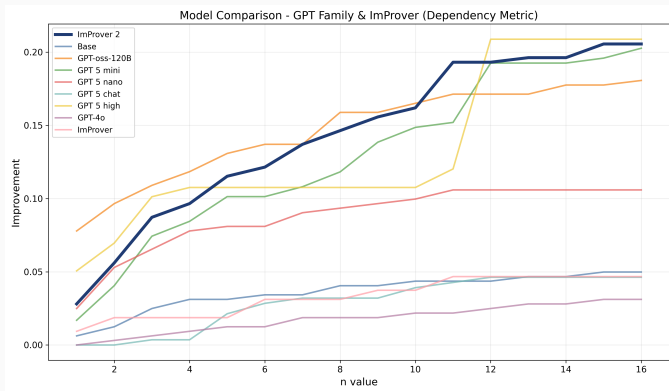
ImProver 2 vs. frontier GPT models, best@ n ($n = 1-16$), length metric.

Frontier Comparison: Modularity (Best@ n)



ImProver 2 leads all models on modularity at every n , including GPT-5-high.

Frontier Comparison: Dependencies (Best@ n)



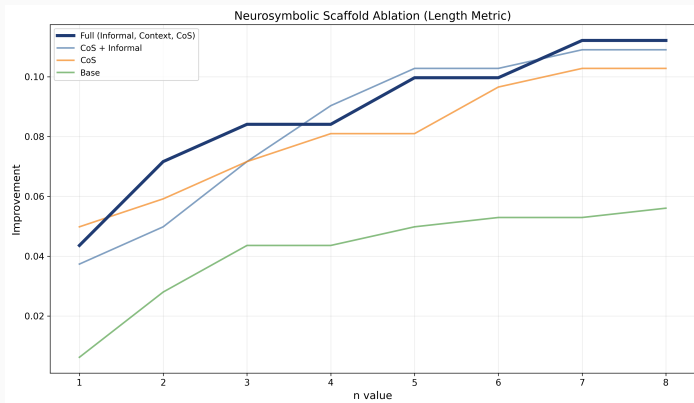
Competitive with frontier on dependencies; gap narrows at higher n .

Effect of Neurosymbolic Scaffold

Metric	Variant	DS-R1 7B	DS-R1 14B	DS-R1 671B	GPT-4o	GPT-oss	GPT-5-nano	GPT-5-mini	GPT-5-chat	GPT-5-high
Length	Base	0.118	0.140	0.308	0.336	0.321	0.087	0.330	0.346	0.660
	Scaffold	0.236	0.202	0.355	0.396	0.508	0.296	0.632	0.576	0.875
Mod.	Base	0.003	0.037	0.055	0.034	0.075	0.065	0.109	0.118	0.120
	Scaffold	0.007	0.024	0.100	0.092	0.092	0.069	0.123	0.153	—
Dep.	Base	0.050	0.093	0.153	0.050	0.181	0.106	0.203	0.046	0.208
	Scaffold	0.056	0.106	0.209	0.075	0.406	0.108	0.267	0.087	0.315

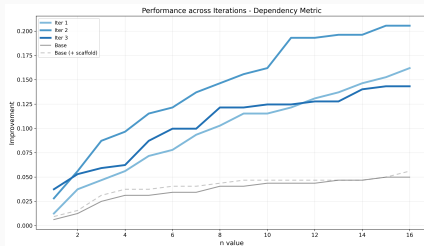
Scaffold improves nearly all models on all metrics. DS-R1-7B length: 0.118 → 0.236 (doubles before any training). GPT-5-high: 0.660 → 0.875. Gains do *not* diminish with scale.

Scaffold Ablation: Each Channel Contributes



Each channel (context slice, CoS, auto-informalization) adds incremental improvement on DS-R1-7B (length metric). Adding each source shows noticeable gains on some n values. Diminishing returns as all three are combined.

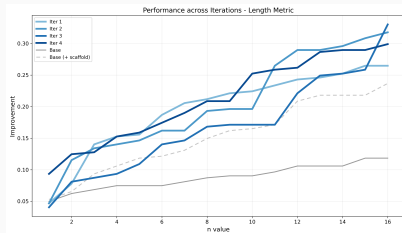
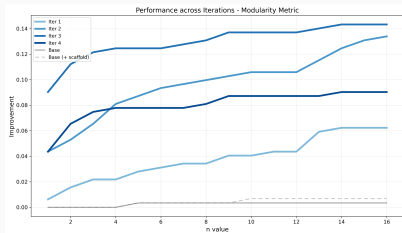
Training and Iteration Results



Iter.	Len	Mod	Dep
Base	0.118	0.003	0.050
+Scaffold	0.236	0.007	0.056
1	0.265	0.062	0.137
2	0.318	0.134	0.206
3	0.330	0.143	0.165
4	0.299	0.096	—

Rapid gains in rounds 1–2. Each metric peaks separately. Round 4: regression = saturation.

Training Iteration: Modularity and Length



Rapid gains in early rounds; saturation and mild regression after peak iteration on all three metrics.

Per-Repository Heterogeneity

Repository	Length	Dep.	Mod.
HEPLearn	1.283	0.379	0.030
ConNF	0.420	0.278	0.048
Seymour	0.199	0.100	0.359
FLT	0.131	0.133	0.066
Foundation	0.082	0.015	0.219
Carleson	0.048	0.188	0.095
Mathlib	0.016	0.306	0.163

Per-Repository Heterogeneity: Interpretation

- **Mathlib**: proofs already concise (low length gain) but admit structural refactoring (high dep./mod. — already well-optimized for length, still room for modularity)
- **HEPLean/ConNF**: large length/dependency gains — proofs have more compression opportunity; less emphasis on existing automation
- **Seymour**: highest modularity gain (0.359) — proof style admits natural decomposition into independent subgoals
- Results are descriptive; reflect stylistic differences and theorem difficulty distributions across repos

Qualitative: Dependency Reduction

isCoatom_iff (Mathlib.Order.Atoms) — 5 named deps $\rightarrow$ 2

Original

```
theorem isCoatom_iff
  [OrderTop A] {K : A} :
  IsCoatom K  $\leftrightarrow$ 
  K  $\neq$  T  $\wedge \forall H g, K \leq H$ 
 $\rightarrow g \notin K \rightarrow g \in H \rightarrow H = T$ 
:= by
simp_rw [IsCoatom,
  lt_iff_le_not_le,
  SetLike.not_le_iff_exists,
  and_comm (a := _  $\leq$  _),
  and_imp, exists_imp,
 $\leftarrow$  and_imp, and_comm]
```

ImProver 2

```
theorem isCoatom_iff
  [OrderTop A] {K : A} :
  IsCoatom K  $\leftrightarrow$ 
  K  $\neq$  T  $\wedge \forall H g, K \leq H$ 
 $\rightarrow g \notin K \rightarrow g \in H \rightarrow H = T$ 
:= by
constructor <=> intro h
<=> simp_all [IsCoatom,
  lt_iff_le_not_le,
  SetLike.not_le_iff_exists]
<=> tauto
```

Automation replaces brittle
named-rewrite chain. 5 explicit
deps $\rightarrow$ 2.

Qualitative: Length Reduction (HEPLearn)

summerCommute_jacobi (HEPLearn.PerturbationTheory) — 43 lines →
24 (**-19 tactics**)

Original

```
lemma summerCommute_jacobi ... := by
  repeat rw [superCommuteF_ofCrAnListF...]
  simp only [instCommGroup, map_sub, ...]
  repeat rw [superCommuteF_ofCrAnListF...]
  simp only [instCommGroup.eq_1, ...]
  by_cases h1 : (FF |>s Phis1) = bosonic <|>
    by_cases h2 : (FF |>s Phis2) = bosonic <|>
    by_cases h3 : (FF |>s Phis3) = bosonic
  -- Case 1 of 8: all bosonic
  . simp only [h1, h2, h3,
    bosonic_exchangeSign, one_smul, ...]
  abel
  -- ... (6 more explicit simp/abel cases) ...
  . simp only [neq_bosonic_iff_eq_fermionic]
    at h1 h2 h3
  simp only [h1, h2, h3, mul_self, ...]
  abel
```

ImProver 2

```
lemma summerCommute_jacobi ... := by
  simp_all [
    superCommuteF_ofCrAnListF...,
    instCommGroup, map_sub, map_smul,
    instCommGroup.eq_1,
    ofList_append_eq_mul,
    neq_bosonic_iff_eq_fermionic]
  <|> by_cases h1 : ... = bosonic
  <|> by_cases h2 : ... = bosonic
  <|> by_cases h3 : ... = bosonic
  <|> simp_all [h1, h2, h3,
    bosonic_exchangeSign,
    fermionic_exchangeSign_fermionic,
    bosonic_mul_fermionic, ...]
  <|> abel
```

8 explicit bosonic/fermionic
case branches unified into
single simp_all + abel pass. 105

Qualitative: Modularity Increase

KD_weakerThan_KDB (Seymour) — flat proof $\rightarrow$ 2 effective named subgoals

Original

```
lemma KD_weakerThan_KDB :  
  (Hilbert.KD  $\alpha$ )  $\leq_s$   
  (Hilbert.KDB  $\alpha$ ) :=  
  
  normal_weakerThan_of_subset  
  (by intro; aesop)
```

Flat; modularity = 0.

ImProver 2

```
lemma KD_weakerThan_KDB :  
  (Hilbert.KD  $\alpha$ )  $\leq_s$   
  (Hilbert.KDB  $\alpha$ ) := by  
  have h1 : KD.axioms  $\subseteq$   
    KDB.axioms ->  
    KD  $\leq_s$  KDB := by  
    apply normal_weakerThan_of_ $\subseteq$   
  have h2 : KD.axioms  $\subseteq$   
    KDB.axioms := by  
    intro phi hphi  
    cases hphi <;>  
    simp_all [KD, KDB]  
  exact h1 h2
```

2 effective spawned goals: h_1, h_2 .

Conclusions

Summary: ImProver

- **Task formalization:** proof optimization as a well-defined task with flexible metric framework; subsumes NTP as a strict special case
- **Chain-of-States:** mechanism for annotating intermediate proof states as generation context; single largest ablation contributor
- **Scaffold:** error correction, refinement loops, and RAG over Mathlib + TPiL
- **Results:** 423–778% over GPT-4o baseline; 100% compilation via fallback guarantee
- **Feasibility:** establishes that agentic prompting with frontier LLMs can produce meaningful proof optimization

Summary: ImProver 2

- **Training pipeline:** iterative self-improvement via IRPO; winner-winner pairs create dense reward signal beyond binary correctness
- **Replay buffer:** prevents model collapse; mixes historical and new data with improvement-rate filtering
- **Neurosymbolic scaffold:** context slice + CoS + auto-informalization consistently improves performance across all model sizes
- **New metrics:** modularity and dependencies with kernel-level reward-hacking guards
- **Results:** 7B model outperforms 671B on all structural metrics; competitive with frontier on dependencies; leads all on modularity

Limitations: ImProver

1. **Cost:** many GPT-4o API calls + document retrieval per theorem; library-scale deployment infeasible at current pricing
2. **Closed-source:** tightly coupled to GPT-4o; non-reproducible; subject to model deprecation
3. **Small evaluation:** only $72 + 26 + 43 = 141$ theorems; toy datasets not representative of real research repositories with inter-file dependencies
4. **Weak metrics:** declarativity gameable via have-padding; no kernel-level structural metric

Limitations: ImProver 2

1. **Unvalidated metrics:** structural metrics (modularity, dependencies) not validated against human-maintainer preferences via a formal study
2. **Single-step generator:** no recovery when rewrite fails — deliberate design choice to study training dynamics before adding agentic complexity
3. **Training saturation:** saturates after 2–4 rounds on fixed dataset; escaping saturation requires larger preference datasets or explicit curriculum growth

1. **Trained iterative revision**

Internalize ImProver's error-correction loop into model weights; train the model to know when to stop, when to revise, and how to incorporate compiler feedback

2. **Human preference study**

Validate modularity/dependency metrics against formal mathematician judgments via a controlled, blinded study

3. **Richer metrics**

LLM-as-judge readability scoring; robustness to library version updates; maintainability proxies

Future Work: Longer-Term

1. **Library-level optimization**

Maintain consistency across inter-dependent Mathlib rewrites; ensure a modular refactor of one theorem does not break downstream proofs referencing its intermediate names

2. **Close the training data loop**

Empirically show that provers trained on ImProver/ImProver 2-optimized corpora outperform those trained on unoptimized data — confirming the data quality hypothesis

3. **Autoformalization quality**

Use proof optimization to make machine-formalized proofs (e.g. AlphaProof outputs) human-readable and maintainable

Closing Remarks

- Formal libraries are growing faster than human maintainers can curate them
- AI-generated proofs will flood these libraries — automated quality improvement is necessary
- **Central insight:** giving a model access to formal proof structure — via CoS at inference time, via neurosymbolic augmentation at training time — enables far more substantive proof transformations
- Proof optimization is feasible, learnable, and applies to both library maintenance and training data quality

ImProver and ImProver 2 establish proof optimization as a foundation for AI-assisted formal mathematics.

Thank you!

ImProver: ICLR 2025

ImProver 2: ICML 2026 (submitted)

Questions welcome.